PERTEMUAN 07

Segmentasi Citra dengan Thresholding

Pengolahan Citra dan Visi Komputer · RTI245007

Tujuan & sub-capaian pembelajaran

Konsep segmentasi

Definisi, posisi dalam alur visi komputer, kriteria homogenitas

Metode thresholding

Global, adaptive, dan ambang otomatis (Otsu)

Pendekatan lanjut

Region-based dan pengelompokan warna (K-Means)

Eksekusi OpenCV

cv2.threshold() dan cv2.adaptiveThreshold() pada kasus riil

Apa yang harus dikuasai setelah pertemuan ini?

1

Menjelaskan

Menghubungkan segmentasi, histogram, dan keputusan threshold.

2

Memilih metode

Global, adaptive, Otsu, atau threshold rentang warna.

3

Mengimplementasikan

Membangun mask dengan OpenCV dan memeriksa parameternya.

Keluaran belajar

Mahasiswa mampu menjelaskan alasan pemilihan metode, bukan hanya menjalankan satu baris kode.

01

Konsep segmentasi

Jembatan antara citra mentah dan informasi bermakna

Apa itu segmentasi?

Membagi citra menjadi beberapa area (region) yang mirip sifat visualnya: intensitas, warna, atau tekstur.

Membagi

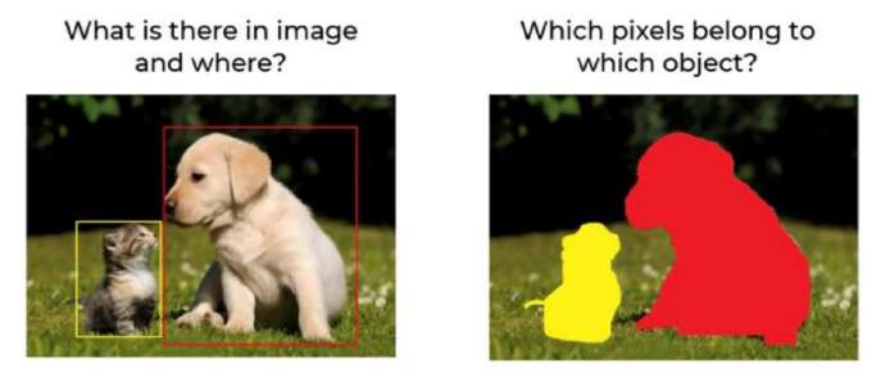
Citra dipecah menjadi region yang memiliki kemiripan sifat visual.

Memisahkan

Objek utama dipisahkan dari latar sehingga bentuk, batas, dan areanya dapat diidentifikasi.

Menyederhanakan

Hasilnya segmen-segmen yang bisa dianalisis terpisah: dasar pengukuran fitur, deteksi, dan klasifikasi.



Deteksi objek menjawab apa dan di mana (kotak). **Segmentasi** menjawab piksel mana milik objek mana.

Ciri area tersegmentasi dan contoh aplikasinya

Ciri umum area

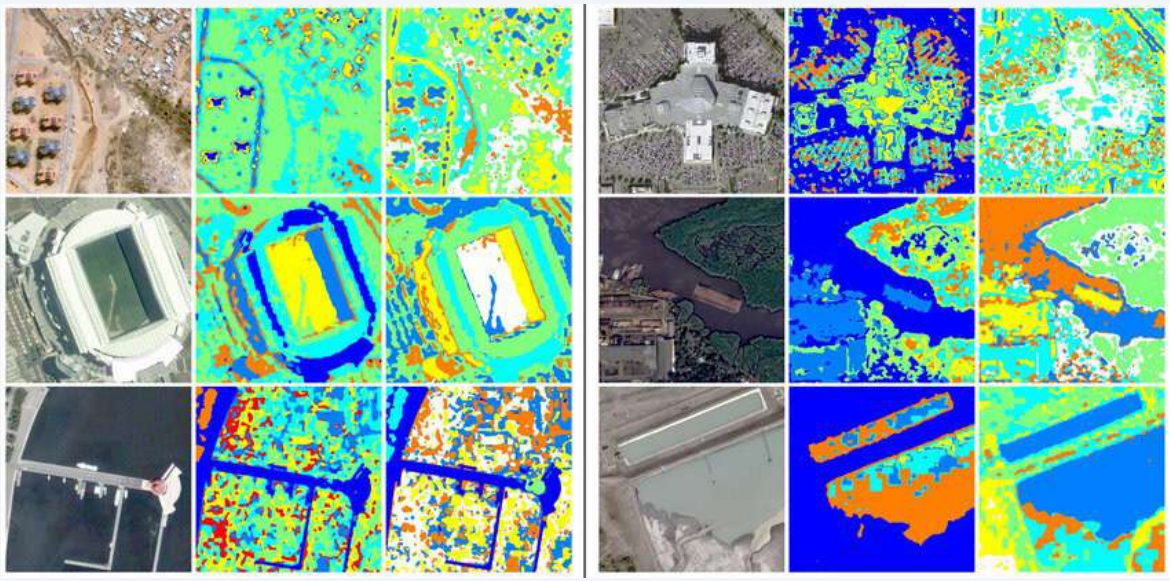
1. Piksel dalam satu area memiliki intensitas yang relatif homogen.
2. Batas antar area menunjukkan perubahan signifikan intensitas, warna, atau tekstur.
3. Setiap area mewakili satu objek, atau bagian objek, yang bermakna dalam konteks tertentu.

Contoh aplikasi

Kesehatan	Memisahkan tumor dari jaringan sehat pada MRI / CT-Scan.
Pertanian	Menghitung daun, buah, atau benih ikan lele dari kamera.
Industri	Inspeksi kualitas produk: deteksi cacat permukaan.
Keamanan	Wajah, plat kendaraan, atau objek mencurigakan di CCTV.
Lingkungan	Area hijau, tanah, atau air pada citra satelit.

Segmentasi bukan hasil akhir. Ia tahap awal analisis: kualitasnya menentukan ekstraksi fitur dan klasifikasi. Metode dipilih sesuai karakteristik citra.

Contoh: segmentasi citra udara



Seperti menggambar peta

Segmentasi menggambar batas antar wilayah berdasarkan warna atau tekstur yang mirip, agar komputer tahu mana bagian kota, gunung, dan laut.

Kiri pada tiap pasangan: foto udara asli. Kanan: hasil segmentasi, satu warna per jenis wilayah.

Posisi segmentasi dalam visi komputer

Segmentasi mengubah citra mentah (level rendah) menjadi representasi berbasis objek (level menengah).

1
Input citra

Matriks piksel RGB / grayscale dari kamera.

›

2
Preprocessing

Reduksi noise, kontras, perbaikan kualitas.

›

3
Segmentasi

Pemisahan objek (foreground) vs latar (background).

›

4
Ekstraksi fitur

Bentuk, luas, warna, dan tekstur.

›

5
Klasifikasi

Pengenalan objek (ML / deep learning).

Sebelum segmentasi

Kualitas hasil sangat bergantung pada preprocessing. Noise tinggi atau kontras rendah sering menghasilkan segmentasi buruk.

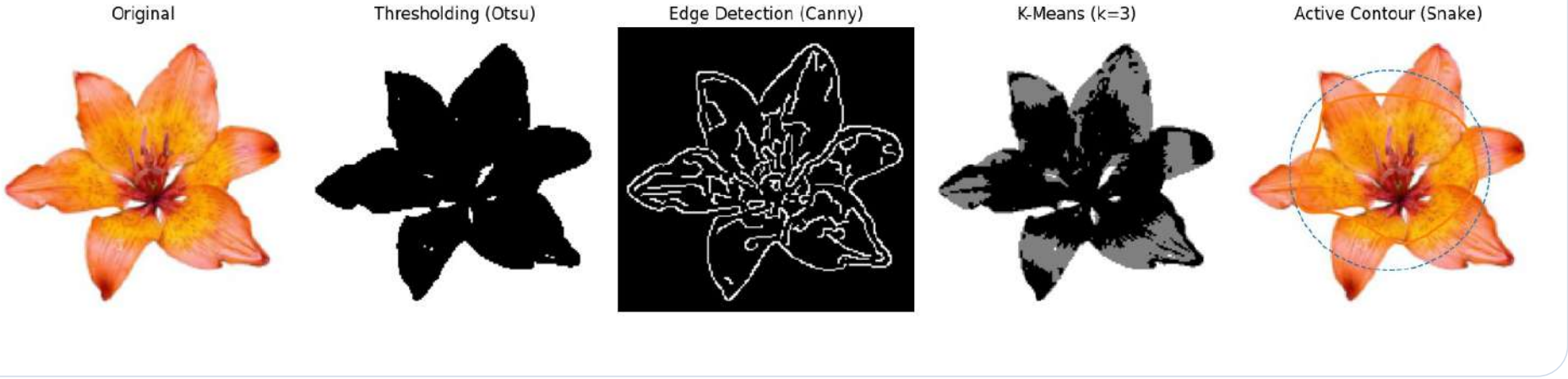
Sesudah segmentasi

Hasil segmentasi menjadi dasar perhitungan fitur: luas, perimeter, bentuk. Kesalahan di sini merambat ke analisis berikutnya.

Taksonomi metode segmentasi citra

Kategori	Prinsip utama	Teknik populer	Skenario terbaik
Thresholding	Memisahkan piksel berdasarkan nilai ambang T	Global, Adaptive, Otsu	Citra kontras tinggi
Edge-based	Mendeteksi diskontinuitas / perubahan gradien intensitas yang mendadak	Sobel, Canny, Prewitt	Batas atau garis objek yang tegas
Region-based	Mengelompokkan piksel tetangga yang memenuhi syarat kesamaan	Region Growing, Splitting	Objek dengan area intensitas seragam
Clustering	Partisi ruang fitur piksel (RGB/HSV) tanpa supervisi	K-Means, Mean-Shift	Citra berwarna kompleks, multi-objek
Deep Learning	Belajar fitur spasial dan kategori piksel secara end-to-end	U-Net, Mask R-CNN, SegNet	Medis dan kendaraan otonom skala besar

Satu citra, hasil berbeda untuk tiap metode



Thresholding
Objek vs latar, dua kelas.

Edge (Canny)
Hanya garis batas.

K-Means
Beberapa kelompok warna.

Active contour
Kurva yang menyusut ke tepi objek.

Panduan memilih algoritma segmentasi

Kondisi citra input	Metode yang disarankan	Alasan
Pencahayaan rata, kontras tinggi, bimodal	Global / Otsu threshold	Komputasi sangat cepat, pemisahan tegas.
Bayangan gradien / pencahayaan tidak merata	Adaptive Gaussian threshold	Ambang menyesuaikan piksel tetangga lokal.
Citra berwarna, multi-objek, kompleks	K-Means clustering	Memakai vektor [R, G, B] untuk partisi warna.
Citra medis (MRI/CT-Scan) bertekstur halus	Region growing / U-Net	Menjaga keterhubungan struktur jaringan.

Kuncinya: kenali dulu kondisi citranya, baru pilih algoritmanya.

02 Thresholding

Paling sederhana, cocok untuk citra kontras tinggi



Thresholding (pengambangan)

Mengubah citra grayscale menjadi citra biner berdasarkan satu atau lebih nilai ambang T.

$$g(x,y) = \begin{cases} 1, & \text{jika } f(x,y) > T \\ 0, & \text{jika } f(x,y) \leq T \end{cases}$$

T adalah batas keputusan. Untuk citra 8-bit, f(x,y) ∈ [0, 255]; kelas 1 biasa disimpan sebagai 255 (putih).



f(x,y): citra asal

g(x,y): hasil biner

`f(x,y)` : intensitas piksel asal di (x,y)

`T` : nilai ambang, 0 sampai 255

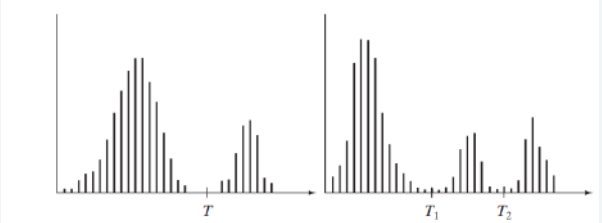
`g(x,y)` : hasil biner 0 atau 1 (255)

Mask bukan sekadar gambar hitam-putih: mask menyatakan piksel mana yang termasuk objek.

Metode thresholding yang umum digunakan

Metode	Ciri khas	Cocok untuk
Global	Satu nilai T	Citra seragam
Adaptive	T lokal per area	Pencahayaan tidak merata
Otomatis (Otsu)	T dihitung otomatis	Citra bimodal
Warna	Berdasarkan kanal warna	Citra berwarna
Gradien	Berdasarkan tepi intensitas	Citra dengan batas tegas
Variabel tertentu	Disesuaikan domain	Citra medis / industri
Adaptif berbasis tepi	Gabungan adaptif & tepi	Segmentasi kompleks

Histogram membantu menilai threshold



Kiri: satu ambang T. Kanan: dua ambang T_1, T_2.

Dua puncak terpisah → kandidat global/Otsu.

Puncak tumpang tindih → keputusan ambigu.

Histogram tidak menyimpan posisi piksel: selalu periksa mask secara spasial. Pilih metode dari karakteristik citra, bukan dari nama algoritmanya.

Global threshold

Satu nilai T untuk seluruh citra. Sederhana, tetapi sensitif terhadap pencahayaan dan pilihan T.



Asli



T = 60



T = 120



T = 180

Di OpenCV

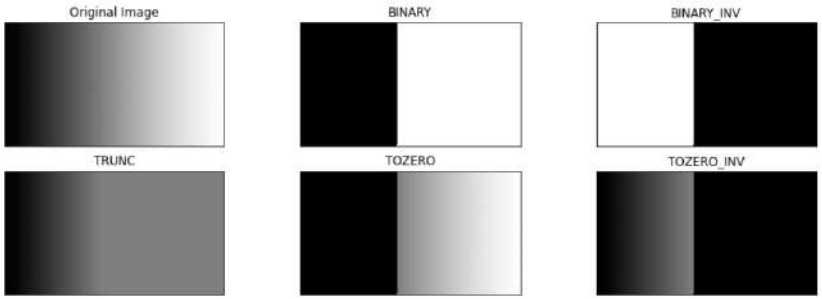
```
nilai_ambang, hasil = cv2.threshold(
    citra_gray, 127, 255,
    cv2.THRESH_BINARY)
```

Coba amati

T yang berbeda menghasilkan mask yang sangat berbeda. T mana yang paling tepat untuk memisahkan juru kamera dari latar?

cv2.threshold(src, thresh, maxval, type)

Parameter	Penjelasan
src	Citra masukan, umumnya grayscale.
thresh	Nilai ambang, misalnya 127.
maxval	Nilai untuk piksel yang lolos, biasanya 255.
type	Aturan thresholding (lihat tabel kanan).



type	piksel > thresh	piksel ≤ thresh
THRESH_BINARY	maxval	0
THRESH_BINARY_INV	0	maxval
THRESH_TRUNC	thresh	tetap
THRESH_TOZERO	tetap	0
THRESH_TOZERO_INV	0	tetap

Adaptive thresholding (local thresholding)

Efektif untuk citra dengan pencahayaan tidak seragam.

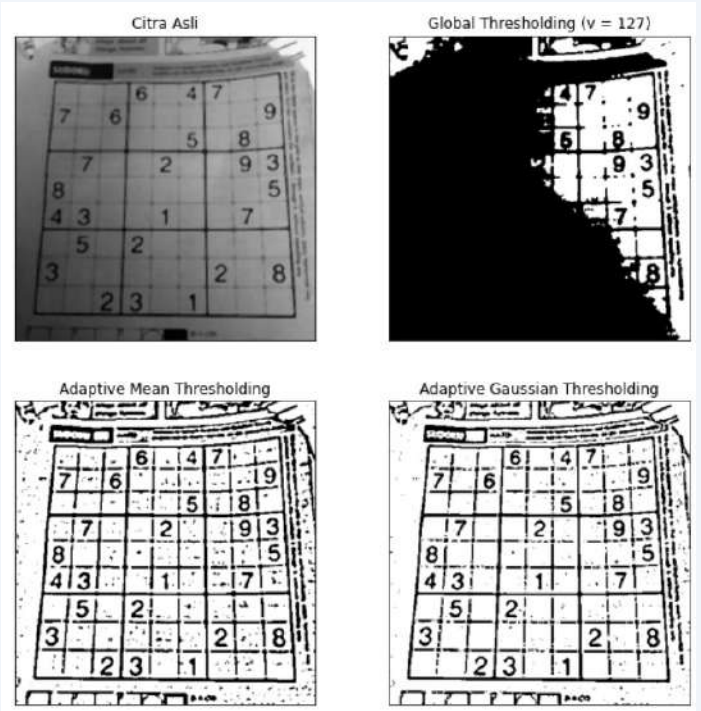
Keterbatasan global threshold

Gagal ketika pencahayaan tidak merata (non-uniform illumination) atau ada bayangan gradien. Satu nilai T tidak bisa mewakili seluruh citra.

Solusi: ambang lokal

Nilai threshold dihitung berbeda untuk setiap piksel, dari area tetangganya (blockSize × blockSize), lalu dikurangi konstanta C.

```
T(x,y) = rata-rata tetangga − C
g(x,y) = 255 jika f(x,y) > T(x,y)
```



Asli, global (T=127), adaptive mean, adaptive Gaussian

cv2.adaptiveThreshold(...)

Parameter	Artinya
src	Citra grayscale, satu kanal, 8-bit.
maxValue	Nilai piksel yang memenuhi syarat; biasanya 255.
adaptiveMethod	ADAPTIVE_THRESH_MEAN_C: rata-rata tetangga. ADAPTIVE_THRESH_GAUSSIAN_C: rata-rata berbobot Gaussian.
thresholdType	THRESH_BINARY atau THRESH_BINARY_INV.
blockSize	Ukuran area tetangga, misal 11 (11×11). Harus ganjil dan > 1.
C	Konstanta yang dikurangkan dari rata-rata lokal, misal 2.

Contoh deklarasi

```
hasil = cv2.adaptiveThreshold(
  citra_gray,      # src
  255,             # maxValue
  cv2.ADAPTIVE_THRESH_MEAN_C,
                   # adaptiveMethod
  cv2.THRESH_BINARY,
                   # thresholdType
  11,              # blockSize
  2)               # C
```

Otsu's thresholding (automatic thresholding)

Cocok untuk citra bimodal: dua puncak intensitas yang jelas.

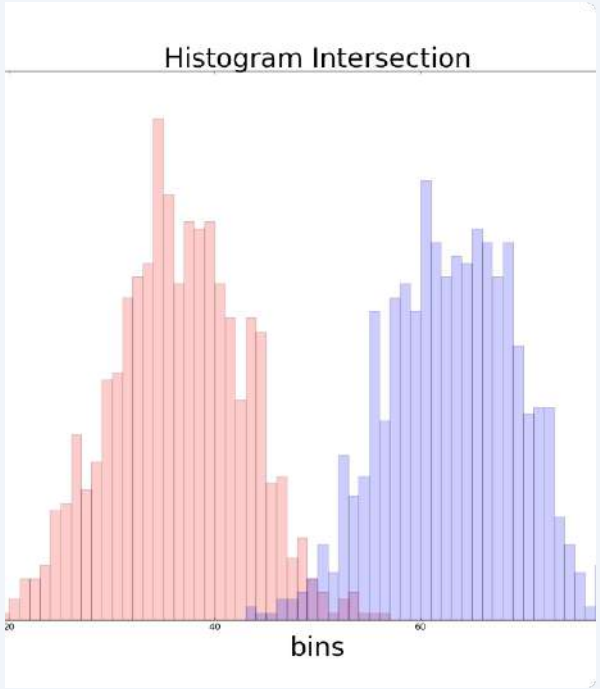
Prinsip kerja: mencari T yang meminimalkan variansi dalam kelas (intra-class), setara dengan memaksimalkan variansi antar kelas foreground dan background. Nobuyuki Otsu, 1979.

$$\sigma_w^2(t) = \omega_0(t)\,\sigma_0^2(t) + \omega_1(t)\,\sigma_1^2(t)$$

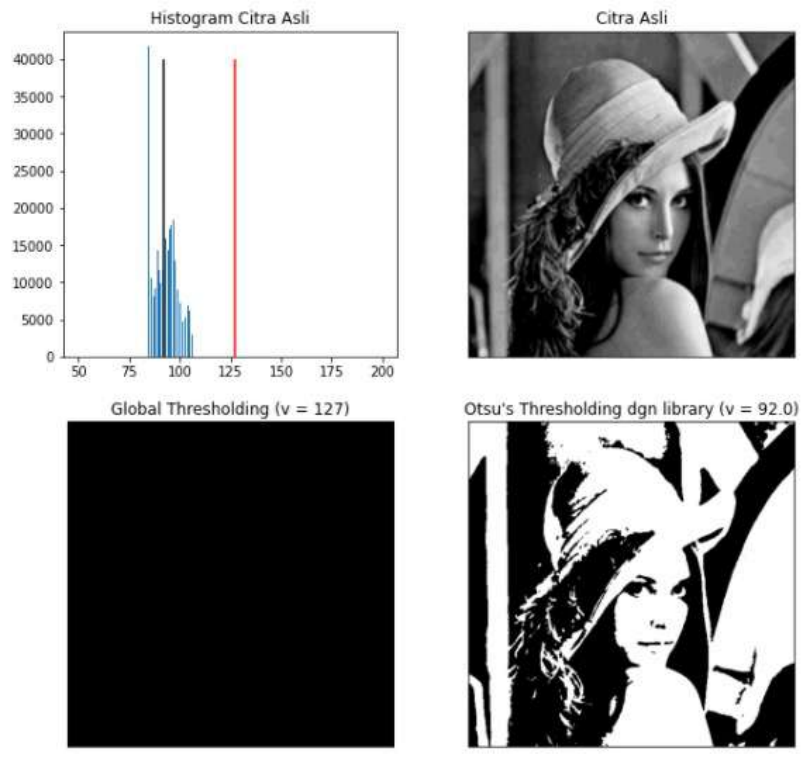
ω_0, ω_1: probabilitas kumulatif dua kelas yang dipisah ambang t

σ_0^2, σ_1^2: variansi intensitas masing-masing kelas

```
nilai_otsu, hasil = cv2.threshold(img, 0, 255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```



Otsu memilih T dengan memaksimalkan pemisahan kelas



Tujuan

Maksimalkan variansi antar kelas

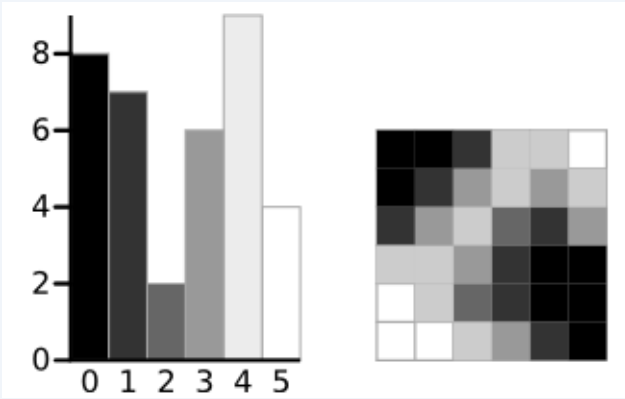
Setara dengan

Minimalkan variansi dalam kelas

Histogram citra ini menumpuk di bawah 127, sehingga global T = 127 membuat hampir semua piksel hitam. Otsu menemukan T = 92.

Asumsi utama: histogram kira-kira bimodal, pencahayaan cukup seragam.

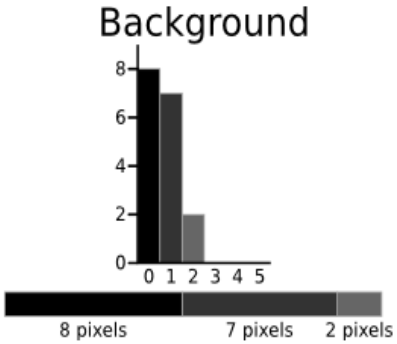
Contoh hitung Otsu: citra 6×6, T = 3



Jumlah piksel per nilai intensitas

0	1	2	3	4	5
8	7	2	6	9	4

Biru = background (< 3), oranye = foreground (≥ 3). Total 36.



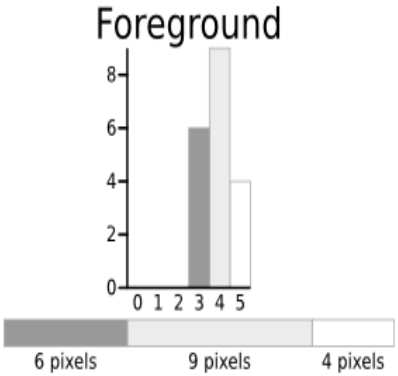
$$\text{Weight} \quad W_b = \frac{8+7+2}{36} = 0.4722$$

$$\text{Mean} \quad \mu_b = \frac{(0\times 8)+(1\times 7)+(2\times 2)}{17} = 0.6471$$

$$\text{Variance} \quad \sigma_b^2 = \frac{((0-0.6471)^2\times 8)+((1-0.6471)^2\times 7)+((2-0.6471)^2\times 2)}{17}$$

$$= \frac{(0.4187\times 8)+(0.1246\times 7)+(1.8304\times 2)}{17}$$

$$= 0.4637$$



$$\text{Weight} \quad W_f = \frac{6+9+4}{36} = 0.5278$$

$$\text{Mean} \quad \mu_f = \frac{(3\times 6)+(4\times 9)+(5\times 4)}{19} = 3.8947$$

$$\text{Variance} \quad \sigma_f^2 = \frac{((3-3.8947)^2\times 6)+((4-3.8947)^2\times 9)+((5-3.8947)^2\times 4)}{19}$$

$$= \frac{(4.8033\times 6)+(0.0997\times 9)+(4.8864\times 4)}{19}$$

$$= 0.5152$$

Within-class variance untuk T = 3

```
σ²_W = W_b·σ²_b + W_f·σ²_f
     = 0,4722·0,4637 + 0,5278·0,5152 = 0,4909
```

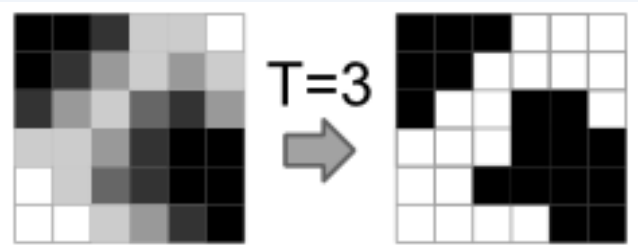
Coba semua T, lalu pilih yang terbaik

Threshold	T = 0	T = 1	T = 2	T = 3	T = 4	T = 5
Within-class σ^2_W	3,1196	1,5268	0,5561	0,4909	0,9779	2,2491
Between-class σ^2_B	0,0000	1,5928	2,5635	2,6287	2,1417	0,8705

Hubungan keduanya

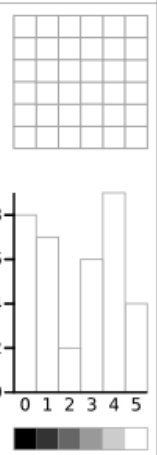
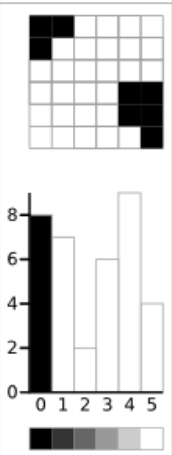
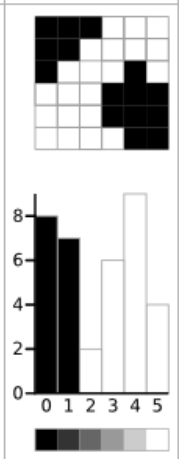
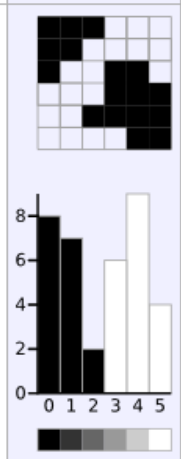
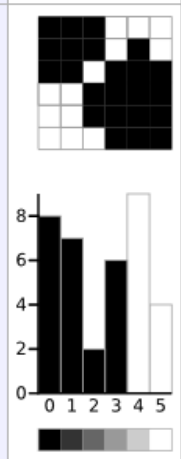
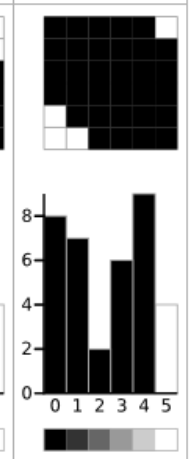
$$\sigma_B^2 = \sigma^2 - \sigma_W^2 = W_b W_f (\mu_b - \mu_f)^2$$

σ^2 = 3,1196 (variansi total) tetap untuk semua T. Jadi T dengan σ^2_W terkecil pasti juga punya σ^2_B terbesar: **T = 3.**



Citra 6×6 sebelum dan sesudah T = 3

Kesalahan threshold: under- dan over-segmentation

Threshold	T=0	T=1	T=2	T=3	T=4	T=5
						
Weight, Background	$W_b = 0$	$W_b = 0.222$	$W_b = 0.4167$	$W_b = 0.4722$	$W_b = 0.6389$	$W_b = 0.8889$
Mean, Background	$M_b = 0$	$M_b = 0$	$M_b = 0.4667$	$M_b = 0.6471$	$M_b = 1.2609$	$M_b = 2.0313$
Variance, Background	$\sigma^2_b = 0$	$\sigma^2_b = 0$	$\sigma^2_b = 0.2489$	$\sigma^2_b = 0.4637$	$\sigma^2_b = 1.4102$	$\sigma^2_b = 2.5303$
Weight, Foreground	$W_f = 1$	$W_f = 0.7778$	$W_f = 0.5833$	$W_f = 0.5278$	$W_f = 0.3611$	$W_f = 0.1111$
Mean, Foreground	$M_f = 2.3611$	$M_f = 3.0357$	$M_f = 3.7143$	$M_f = 3.8947$	$M_f = 4.3077$	$M_f = 5.000$
Variance, Foreground	$\sigma^2_f = 3.1196$	$\sigma^2_f = 1.9639$	$\sigma^2_f = 0.7755$	$\sigma^2_f = 0.5152$	$\sigma^2_f = 0.2130$	$\sigma^2_f = 0$
Within Class Variance	$\sigma^2_W = 3.1196$	$\sigma^2_W = 1.5268$	$\sigma^2_W = 0.5561$	$\sigma^2_W = 0.4909$	$\sigma^2_W = 0.9779$	$\sigma^2_W = 2.2491$

T terlalu rendah

Terlalu banyak piksel masuk foreground → false positive.

T terlalu tinggi

Bagian objek hilang → false negative.

Nilai T terbaik adalah yang bekerja pada tujuan dan data uji, bukan mask yang "terlihat bagus" pada satu citra.

Otsu bukan "selalu optimal"

Otsu hanya optimal untuk kriterianya sendiri: memisahkan dua kelompok intensitas.

Baik

Foreground dan background membentuk dua kelompok intensitas.

Lemah

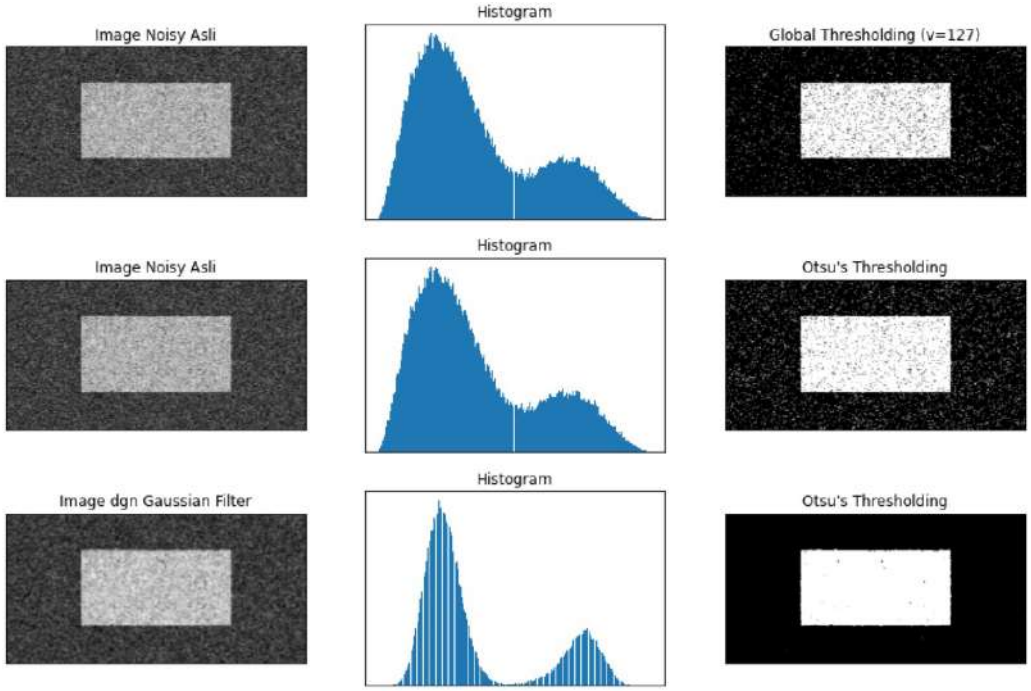
Ukuran kelas sangat tidak seimbang, atau puncak bertumpuk.

Lemah

Gradien pencahayaan kuat: pertimbangkan adaptive threshold.

Lebih dari 2 kelas

Multi-Otsu menghasilkan beberapa threshold.



Tips: Gaussian blur sebelum Otsu sering menekan noise; efeknya tetap harus divalidasi.

03 Pendekatan lanjut

Segmentasi region-based dan berbasis clustering

Region-based segmentation

Membagi citra menjadi area homogen berdasarkan kesamaan intensitas, warna, atau tekstur.

Ciri pendekatan

1. Fokus pada kesamaan antar piksel tetangga.
2. Menekankan keterhubungan area, bukan batas tepi.
3. Biasanya diawali dari titik awal (seed) lalu meluas.
4. Cocok bila kontras tidak tegas tetapi area objek homogen.

Kriteria homogenitas

Region dianggap homogen jika salah satu terpenuhi:

- Selisih intensitas antar piksel < ambang tertentu.
- Selisih warna di ruang RGB/HSV < batas tertentu.
- Variansi intensitas dalam region < nilai tertentu.
- Tekstur lokal seragam (misal GLCM energy tinggi).

Bedanya dengan thresholding: thresholding melihat nilai tiap piksel sendiri-sendiri; region-based melihat kemiripan piksel yang berdekatan.

Jenis metode segmentasi berbasis wilayah

Metode	Prinsip	Kelebihan	Kekurangan
Region Growing	Menumbuhkan area dari titik seed dengan menambahkan piksel tetangga bernilai mirip.	Sederhana, hasil halus	Sensitif terhadap seed & noise
Splitting & Merging	Membagi citra menjadi blok kecil, lalu menggabungkan blok yang serupa.	Cocok untuk citra kompleks	Lambat untuk citra besar
Watershed	Citra sebagai topografi (ketinggian = intensitas); memisahkan lembah menjadi region.	Presisi di batas objek	Over-segmentation tanpa pra-proses
Mean Shift	Mengelompokkan piksel berdasarkan kepadatan fitur (warna & posisi).	Adaptif, tanpa jumlah cluster	Berat secara komputasi

1. Region growing

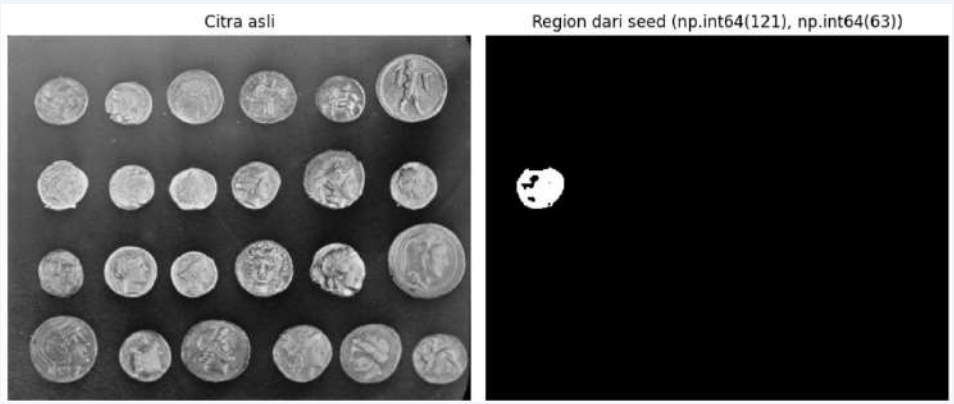
Mulai dari piksel seed, lalu tambahkan tetangga yang mirip sampai tidak ada lagi yang memenuhi.

Langkah

1. Pilih seed.
2. Periksa tetangga piksel yang sudah masuk region (8-ketetanggaan).
3. Jika $|I(\text{tetangga}) - \text{rata-rata region}| \leq T$, masukkan ke region.
4. Ulangi sampai tidak ada piksel baru.

Kelebihan sederhana, region selalu terhubung

Kekurangan sensitif terhadap pilihan seed dan T



Seed di koordinat (121, 63). Region hanya tumbuh pada koin yang memuat seed, karena koin lain tidak terhubung dengannya.

Region growing: pseudocode

```
ALGORITMA RegionGrowing(citra I, daftar seed S, threshold T)
  INPUT : citra grayscale I (H x W), seed S, threshold T
  OUTPUT: mask hasil segmentasi M (H x W)

  M ← matriks nol ukuran H x W
  untuk setiap seed s dalam S:
      queue ← [s]
      M[s] ← 1
      mean_region ← I[s]
      jumlah ← 1

      selama queue tidak kosong:
          p ← ambil_depan(queue)
          untuk setiap tetangga q dari p (4 atau 8 ketetanggaan):
              jika q di dalam batas citra DAN M[q] = 0:
                  jika |I[q] - mean_region| ≤ T:
                      M[q] ← 1
                      tambahkan q ke queue
                      mean_region ← (mean_region × jumlah + I[q]) /
(jumlah + 1)
                      jumlah ← jumlah + 1
  kembalikan M
```

Cara membaca

`queue`
antrean piksel yang menunggu diperiksa tetangganya.

`M`
mask hasil: 1 = masuk region.

`mean_region`
rata-rata region, diperbarui setiap piksel baru masuk.

2. Region splitting and merging

Kebalikan dari region growing: citra awalnya dianggap satu region besar.

Splitting

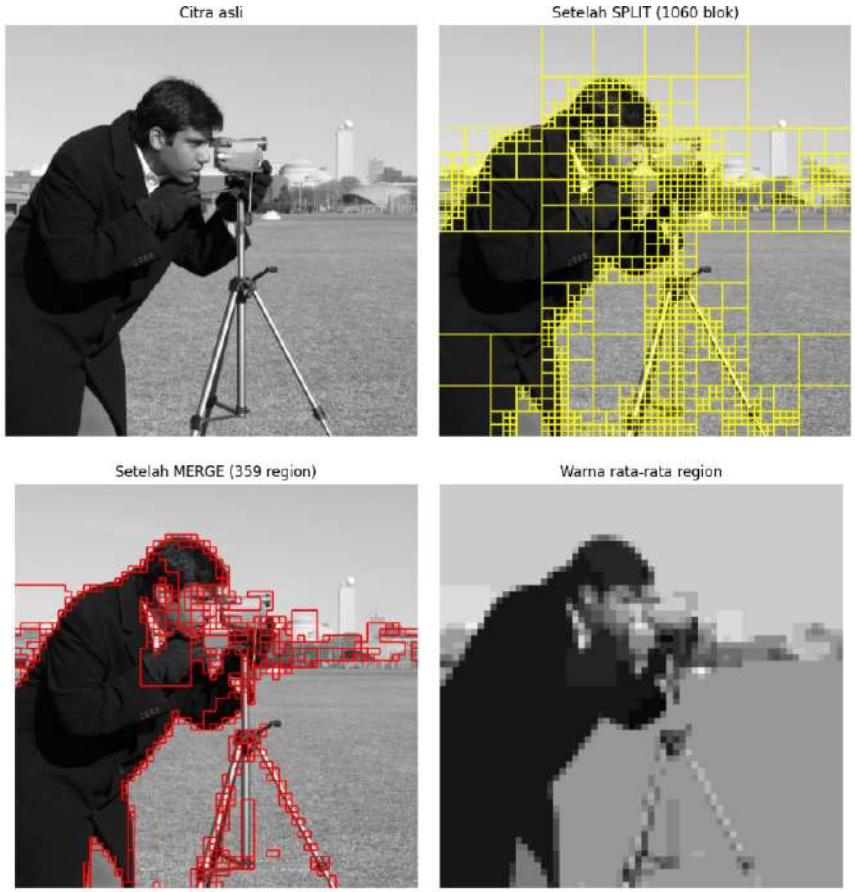
Jika region tidak homogen (misal simpangan baku > T), bagi menjadi 4 kuadran (quadtree). Ulangi rekursif.

Merging

Region bertetangga yang cukup mirip digabung kembali.

Catatan

Tidak perlu seed, tetapi hasil split awal berbentuk blok persegi.



Splitting and merging: pseudocode

```
ALGORITMA SplitAndMerge(citra I, kriteria homogenitas P)
  INPUT : citra I (ukuran 2^n x 2^n), predikat P(R) =
TRUE jika
          (max(R) - min(R)) ≤ T atau std(R) ≤ T
  OUTPUT: label region L

  // ---------- TAHAP 1: SPLITTING ----------
  FUNGSI Split(region R):
      jika P(R) = TRUE ATAU ukuran(R) ≤ ukuran_minimum:
          tambahkan R ke daftar_region
      selain itu:
          bagi R menjadi 4 kuadran R1, R2, R3, R4
          untuk setiap Ri: Split(Ri)

  daftar_region ← []
  Split(seluruh citra I)
```

```
// ---------- TAHAP 2: MERGING ----------
ulangi:
    ada_merge ← FALSE
    untuk setiap pasangan region (Ri, Rj) dalam
daftar_region:
        jika Ri bertetangga dengan Rj DAN P(Ri ∪
Rj) = TRUE:
            gabungkan Ri dan Rj menjadi satu region
            ada_merge ← TRUE
sampai ada_merge = FALSE

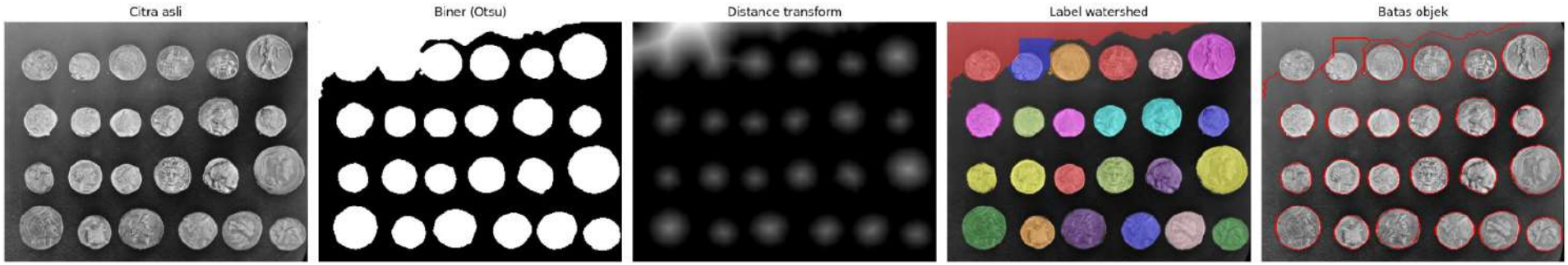
beri label unik pada setiap region akhir ke L
kembalikan L
```

3. Watershed segmentation

Konsep: citra dipandang sebagai permukaan topografi (intensitas/gradien = ketinggian). Air dituang dari setiap marker (cekungan); saat air dari cekungan berbeda bertemu, dibangun "bendungan" yang menjadi batas antar objek.

Masalah klasik: tanpa marker, noise menyebabkan over-segmentation. Solusinya: marker-controlled watershed.

1. Threshold (Otsu) menjadi citra biner.
2. Distance transform: jarak piksel objek ke latar.
3. Puncak lokal menjadi marker, satu per objek.
4. Watershed pada –distance dengan marker.



Watershed: pseudocode

```
ALGORITMA Watershed(citra I)
  INPUT : citra I
  OUTPUT: label region L, dengan batas ditandai -1

  // ---------- PRA-PROSES ----------
  G  ← grayscale(I)
  B  ← threshold_otsu(G)  // biner: objek vs latar
  B  ← opening_morfologi(B)  // buang noise kecil
  D  ← distance_transform(B)  // jarak ke piksel latar
terdekat

  // ---------- PEMBUATAN MARKER ----------
  sure_fg ← threshold(D, 0.5 × max(D))  // inti objek
pasti
  sure_bg ← dilasi(B)  // area latar pasti
  unknown ← sure_bg − sure_fg
  markers ← connected_components(sure_fg) // beri label
1..N tiap inti
  markers ← markers + 1  // latar = 1, objek = 2..N+1
  markers[unknown] ← 0  // area belum ditentukan
```

```
// ---------- FLOODING ----------
PQ ← priority_queue berdasarkan nilai gradien
masukkan semua piksel tepi marker ke PQ

selama PQ tidak kosong:
    p ← ambil piksel dengan prioritas terendah
    label_tetangga ← kumpulan label marker dari
tetangga p
    jika semua label_tetangga sama (satu label):
        markers[p] ← label tersebut
        masukkan tetangga p yang belum berlabel
ke PQ
    jika terdapat label berbeda:
        markers[p] ← −1  // piksel batas (garis
watershed)

kembalikan markers
```

4. Mean shift segmentation

Konsep: setiap piksel menjadi titik di ruang fitur (warna + posisi). Tiap titik digeser berulang kali ke rata-rata titik di sekitarnya (dalam radius bandwidth) sampai konvergen ke puncak kepadatan (mode). Piksel yang konvergen ke mode yang sama menjadi satu segmen.

Parameter utama

`sp` spatial radius: jangkauan spasial

`sr` color radius: beda warna yang masih dianggap sama

\+ tidak perlu jumlah segmen − komputasi berat



Mean shift: pseudocode

```
ALGORITMA MeanShift(citra I, bandwidth spasial hs, bandwidth
warna hr)
  INPUT : citra I (H x W x 3)
  OUTPUT: citra tersegmentasi S, label L

  // ---------- TAHAP 1: FILTERING (konvergensi ke mode) -----
-----
  untuk setiap piksel p = (x, y) pada I:
      titik ← (x, y, r, g, b)  // ruang fitur 5 dimensi
      ulangi:
          N ← semua piksel q dengan
              jarak_spasial(q, titik) ≤ hs  DAN
              jarak_warna(q, titik)   ≤ hr
          titik_baru ← rata-rata fitur (x, y, r, g, b) dari
seluruh N
          geseran ← ||titik_baru - titik||
          titik ← titik_baru
      sampai geseran < ε ATAU iterasi ≥ iterasi_maks
      mode[p] ← titik  // mode (puncak densitas) piksel p
```

```
// ---------- TAHAP 2: PENGELOMPOKAN ----------
label_berikutnya ← 1
untuk setiap piksel p:
    jika L[p] belum berlabel:
        L[p] ← label_berikutnya
        untuk setiap piksel q lain yang belum
berlabel:
            jika jarak_spasial(mode[p], mode[q]) <
hs
               DAN jarak_warna(mode[p], mode[q]) <
hr:
                L[q] ← label_berikutnya
        label_berikutnya ← label_berikutnya + 1

// ---------- TAHAP 3: REKONSTRUKSI ----------
untuk setiap piksel p:
    S[p] ← warna dari mode[p]

kembalikan S, L
```

Membandingkan empat metode region-based

Aspek	Region Growing	Split & Merge	Watershed	Mean Shift
Pendekatan	Bottom-up: tumbuh dari seed	Top-down (split), lalu bottom-up (merge)	Citra sebagai peta topografi	Mode kepadatan di ruang fitur
Kebutuhan input	Seed + threshold	Kriteria homogenitas	Marker (penanda)	Bandwidth
Perlu seed/marker	Ya (seed)	Tidak	Ya (marker)	Tidak
Cocok untuk	Satu objek homogen (mis. organ pada citra medis)	Citra dengan area homogen luas	Objek yang saling bersentuhan	Citra berwarna, segmentasi umum
Risiko	Bocor / seed salah	Batas berbentuk blok	Over-segmentation tanpa marker	Lambat, sensitif bandwidth

Segmentasi berbasis clustering: K-Means

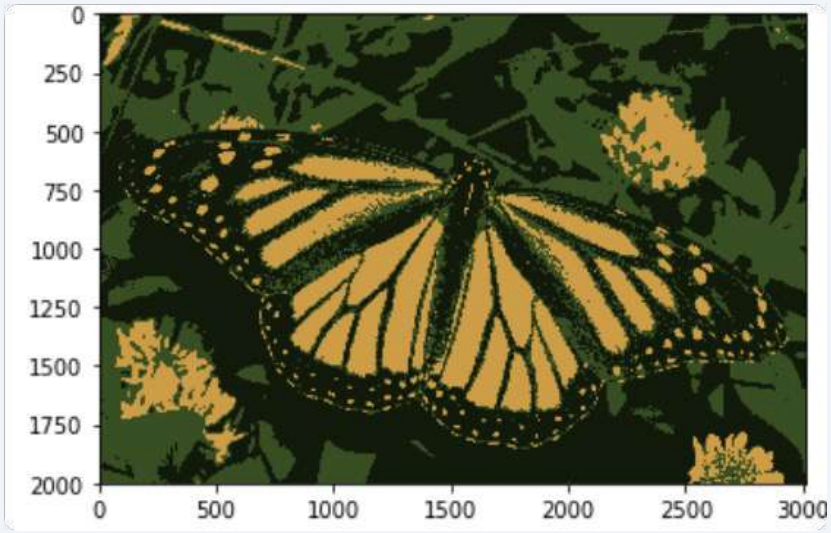
Saat thresholding 1D grayscale tidak cukup, K-Means mengelompokkan piksel berdasarkan warnanya di ruang RGB/HSV.

Centroid terdekat

Setiap piksel masuk ke cluster dengan centroid μ_i terdekat di ruang warna.

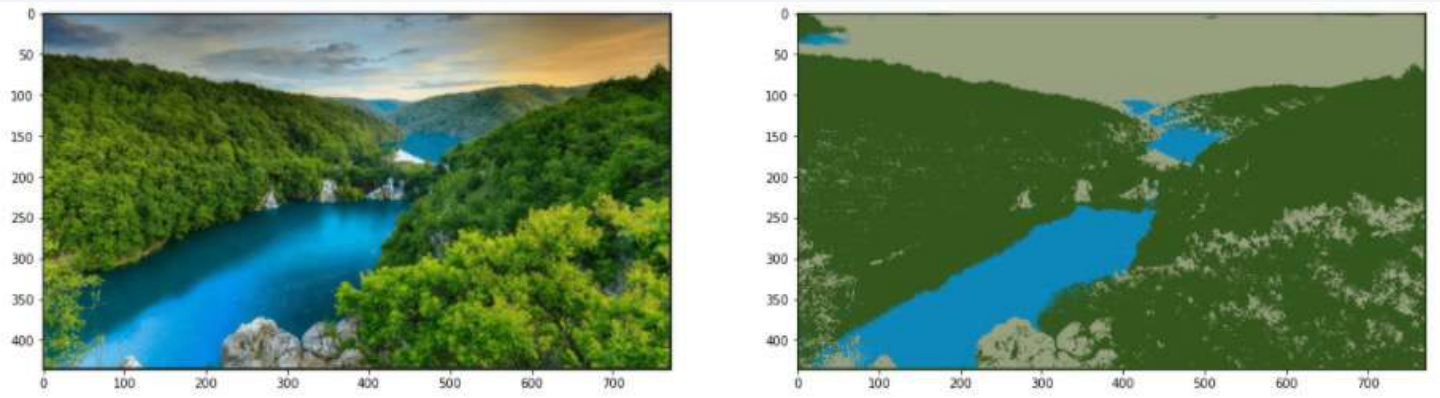
$$J = \sum_{i=1}^{k} \sum_{x_j \in C_i} \|x_j - \mu_i\|^2$$

J: total jarak kuadrat dalam cluster (yang diminimalkan)
$\mathbf{x_j}$: vektor fitur piksel, misal [R, G, B]
$\boldsymbol{\mu_i}$: centroid (rata-rata) cluster ke-i
$\mathbf{C_i}$: himpunan piksel di cluster ke-i



Contoh: kupu-kupu dipisahkan dari daun hanya dengan warna.

Langkah segmentasi citra dengan K-Means



1. Konversi citra ke ruang warna RGB atau HSV.
2. Reshape menjadi array 2D: satu baris = satu piksel.
3. Jalankan K-Means untuk mendapat label cluster.
4. Bentuk ulang label ke dimensi asli citra.
5. Tampilkan setiap cluster sebagai segmen.

Video: youtube.com/watch?v=22mpExWh1LY

Contoh implementasi K-Means

```
import cv2
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Baca citra dan ubah ke RGB
img = cv2.imread('daun.jpg')
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Ubah citra menjadi array 2D (piksel)
Z = img.reshape((-1, 3))
Z = np.float32(Z)

# Tentukan jumlah cluster
k = 3
kmeans = KMeans(n_clusters=k, random_state=42)
kmeans.fit(Z)
```

```
# Dapatkan label dan pusat cluster
labels = kmeans.labels_
centers =
np.uint8(kmeans.cluster_centers_)

# Bentuk citra hasil segmentasi
segmented = centers[labels.flatten()]
segmented_img =
segmented.reshape(img.shape)

# Tampilkan hasil
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1); plt.imshow(img)
plt.title('Citra Asli'); plt.axis('off')
plt.subplot(1, 2, 2);
plt.imshow(segmented_img)
plt.title('Segmentasi K-Means (k=3)')
plt.axis('off')
plt.show()
```

Contoh hasil segmentasi K-Means (k = 3)



Yang terjadi

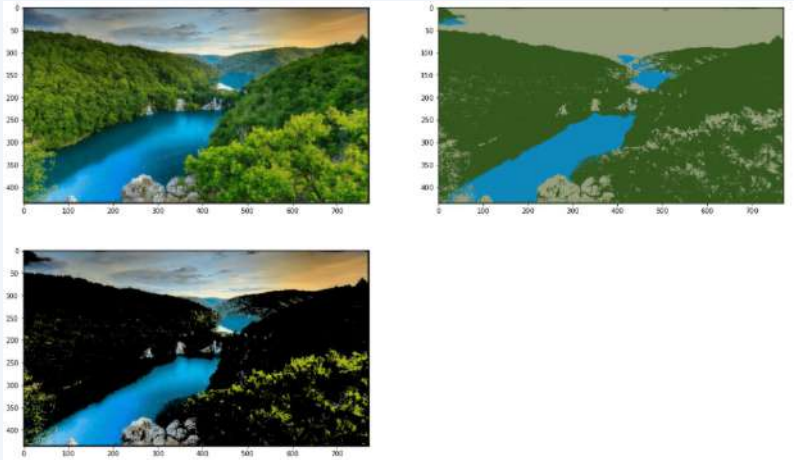
Setiap piksel diganti warna centroid cluster-nya. Bintik dan benang sari hilang karena hanya ada 3 warna.

Coba pikirkan: apa yang berubah jika k dinaikkan menjadi 5?

Percobaan: menghitamkan satu cluster

```
# ubah piksel di cluster 2 menjadi hitam
masked_image = np.copy(img)
# konversi ke bentuk vektor
masked_image = masked_image.reshape((-1, 3))
# cluster yang diubah
cluster = 2
masked_image[labels == cluster] = [0, 0, 0]
# konversi ke bentuk asli
masked_image = masked_image.reshape(img.shape)

plt.figure(figsize=(20, 12))
plt.subplot(2, 2, 1); plt.imshow(img)
plt.subplot(2, 2, 2); plt.imshow(segmented_img)
plt.subplot(2, 2, 3); plt.imshow(masked_image)
```



Asli, hasil K-Means, dan citra dengan satu cluster dihitamkan.

04 Evolusi segmentasi citra

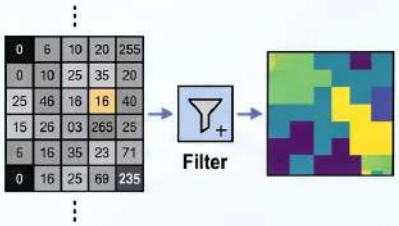
Dari aturan klasik ke model fondasi visual



Evolusi Segmentasi Citra: Dari Aturan Klasik ke Model Fondasi Visual

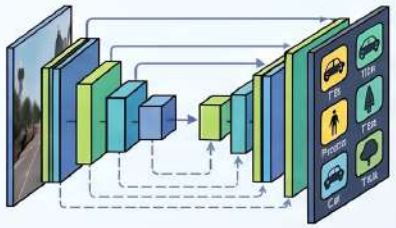
Era Klasik & Konvolusional
(Model Spesifik & Kategori Tertutup)

Pemrosesan Klasik Berbasis Aturan Piksel



Mengandalkan intensitas piksel dan gradien lokal tanpa pemahaman semantik tingkat tinggi.

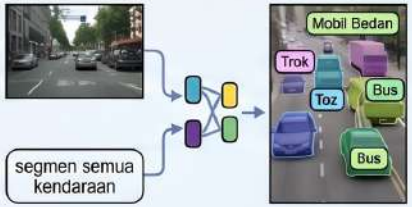
Deep Learning CNN & Asumsi Kategori Tertutup



Arsitektur seperti U-Net dan Mask R-CNN unggul tetapi terbatas pada kelas terlabel.

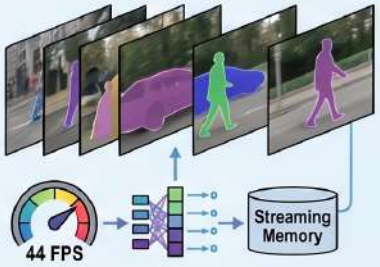
Era Open-Vocabulary & Model Fondasi Visual
(Model Universal & Terbuka)

Integrasi Vision-Language (OVS & LSeg)



Menghubungkan fitur piksel dengan teks untuk mensegmentasi objek tanpa pelatihan ulang.

SAM 2: Inferensi Real-Time 44 FPS



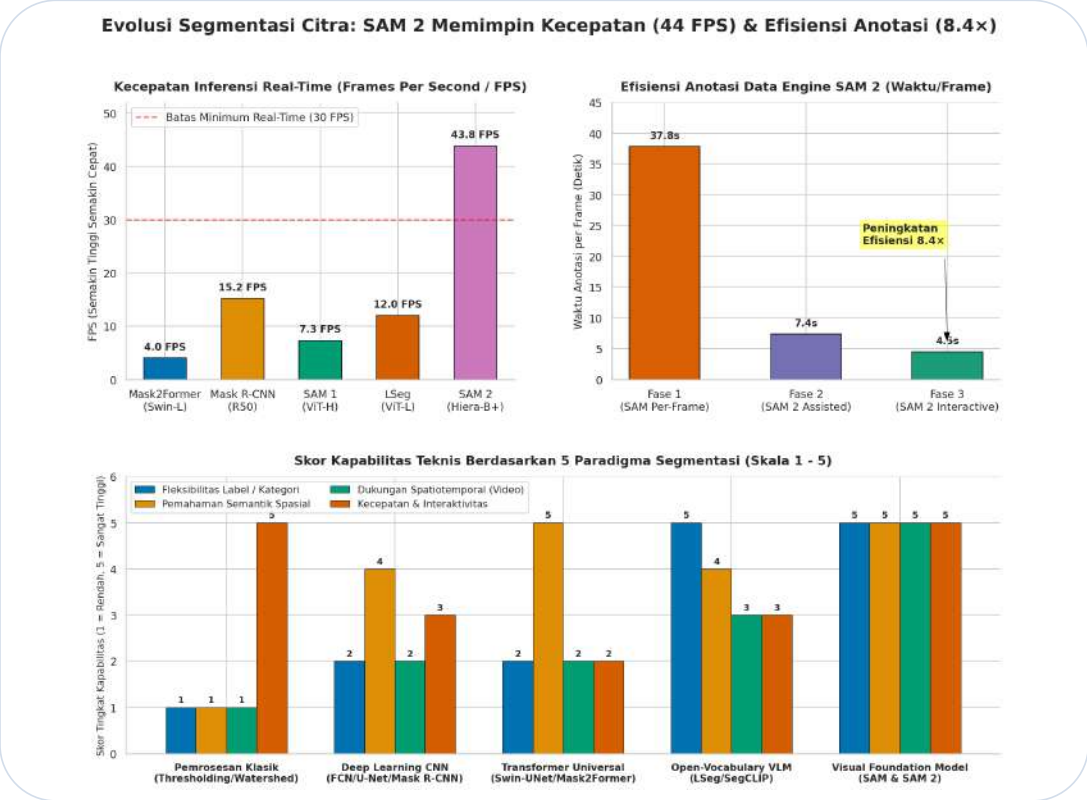
Menggunakan mekanisme streaming memory untuk pelacakan objek spatiotemporal interaktif dalam video.

	Paradigma: Pemrosesan Klasik	Paradigma: Deep Learning CNN	Paradigma: Model Fondasi Visual
Representasi Fitur	**Representasi Fitur:** Intensitas & gradien piksel murni	**Representasi Fitur:** Fitur abstrak spasial hierarkis	**Representasi Fitur:** Fitur spatiotemporal & memory embeddings
Skalabilitas Label	**Skalabilitas Label:** Tanpa label semantik	**Skalabilitas Label:** Closed-set (Kategori terdefinisi)	**Skalabilitas Label:** Class-agnostic / Zero-shot

Video: youtube.com/watch?v=fVeW9a6wItM

Gemini Notebook

SAM 2: kecepatan dan efisiensi anotasi



Yang perlu dicatat

±44 FPS

SAM 2 (Hiera-B+) sudah di atas batas real-time 30 FPS.

37,8 s → 4,5 s

Waktu anotasi per frame turun sekitar 8,4× dengan SAM 2 interaktif.

Skor 1–5

Grafik bawah adalah penilaian kualitatif, bukan hasil benchmark.

Terima kasih!



Team Teaching PCVK